

Spring Data JPA: Advance

Introduction

This section provides a brief recap of fundamental Spring Data JPA concepts and an overview of the advanced topics that will be covered.

Spring Data JPA simplifies the data access layer in Spring applications by reducing the amount of boilerplate code required. It provides a set of interfaces that, when extended by your own repository interfaces, are automatically implemented by Spring at runtime. This allows developers to focus on business logic rather than the intricacies of data access.

Core Concepts Recap:

- **@Entity:** An annotation used to mark a Java class as a JPA entity, representing a table in the database.
- **@Id:** Specifies the primary key of an entity.
- **@GeneratedValue:** Provides for the specification of generation strategies for the values of primary keys.
- **JpaRepository:** An interface that provides JPA-related methods such as flushing the persistence context and deleting records in a batch. It extends the PagingAndSortingRepository and CrudRepository interfaces.
- **Query Methods:** Spring Data JPA can automatically create queries from method names in your repository interfaces. For example, `findByFirstName(String firstName)` will generate a query to find a user by their first name.
- **@Query Annotation:** Allows you to write custom JPQL (Java Persistence Query Language) or native SQL queries in your repository interfaces.

Advanced Topics Overview:

This document will delve into more advanced features of Spring Data JPA, including:

- **Sorting and Pagination:** Efficiently handling large datasets by retrieving data in sorted, manageable chunks.
- **Criteria API:** Building dynamic, type-safe queries at runtime, which is particularly useful for implementing complex search functionalities.
- **Common DAO (Data Access Object):** Creating a generic DAO to further reduce boilerplate code for common data access patterns.

Sorting and Pagination

When dealing with large amounts of data, it's often impractical and inefficient to fetch all records at once. Pagination is the process of dividing a large result set into smaller, discrete pages. Sorting allows you to order the retrieved data based on one or more entity attributes.

Demonstration Scenario: A List of Products

Imagine an e-commerce application with thousands of products. Displaying all products on a single page would lead to poor performance and a bad user experience. Instead, we can display a paginated list of products, perhaps 20 per page, and allow the user to sort them by name, price, or category.

End Application Output:

The end goal is to build a REST API that can accept parameters for pagination and sorting. For instance, a request to `/api/products?page=0&size=10&sort=price,asc` would retrieve the first page of 10 products, sorted by price in ascending order.

The JSON response for such a request would look something like this:

```
{
  "content": [
    // 10 product objects sorted by price
  ],
  "pageable": {
    "sort": {
      "sorted": true,
      "unsorted": false,
      "empty": false
    },
    "offset": 0,
    "pageNumber": 0,
    "pageSize": 10,
    "paged": true,
    "unpaged": false
  },
  "totalPages": 100,
  "totalElements": 1000,
  "last": false,
  "size": 10,
  "number": 0,
  "sort": {
    "sorted": true,
    "unsorted": false,
    "empty": false
  },
}
```

```
"numberOfElements": 10,  
"first": true,  
"empty": false  
}
```

This response not only includes the list of products (content) but also metadata about the pagination, such as the total number of pages and elements.

Pageable Interface

The core of pagination and sorting in Spring Data JPA is the Pageable interface. It's a simple interface that carries pagination information.

Pageable Interface:

The `org.springframework.data.domain.Pageable` interface contains methods to get pagination and sorting details:

- `int getPageNumber():` Returns the current page number.
- `int getPageSize():` Returns the number of items to be returned on a page.
- `long getOffset():` Returns the offset to be taken according to the page number and page size.
- `Sort getSort():` Returns the sorting parameters.

PageRequest:

You don't typically implement the Pageable interface yourself. Instead, you use the PageRequest class, which is a concrete implementation of Pageable. You can create a PageRequest object using its static `of()` methods:

```
// For a paginated request without sorting  
Pageable pageable = PageRequest.of(0, 10); // page 0, size 10  
  
// For a paginated request with sorting  
Pageable sortedPageable = PageRequest.of(0, 10, Sort.by("name")); // sort  
by name ascending  
  
// For sorting in descending order  
Pageable descendingPageable = PageRequest.of(0, 10,  
Sort.by("price").descending());  
  
// For sorting by multiple fields  
Pageable multiSortPageable = PageRequest.of(0, 10,  
Sort.by("category").ascending().and(Sort.by("name").descending()));
```

In a Spring MVC controller, you can directly accept a Pageable object as a method parameter. Spring will automatically create a PageRequest object from the request parameters (page, size, and sort).

```
@GetMapping("/products")
public Page<Product> getAllProducts(Pageable pageable) {
    return productService.findAll(pageable);
}
```

A request to `/products?page=1&size=20&sort=name,desc` will automatically be mapped to a Pageable object.

PagingAndSortingRepository

The PagingAndSortingRepository is an interface in Spring Data that extends CrudRepository to provide methods for retrieving entities using pagination and sorting.

PagingAndSortingRepository Interface:

It provides two main methods:

- **Page<T> findAll(Pageable pageable):** Returns a Page of entities meeting the paging restriction provided in the Pageable object.
- **Iterable<T> findAll(Sort sort):** Returns all entities sorted by the given Sort option.

Implementation Example:

We've already extended **PagingAndSortingRepository** in our **ProductRepository**. Now, let's create a service and a controller to expose this functionality.

Product Service:

```
@Service
public class ProductService {

    @Autowired
    private ProductRepository productRepository;

    public Page<Product> findAllProducts(Pageable pageable) {
        return productRepository.findAll(pageable);
    }
}
```

Product Controller:

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Autowired
    private ProductService productService;

    @GetMapping
    public Page<Product> getAllProducts(Pageable pageable) {
        return productService.findAllProducts(pageable);
    }
}
```

Testing the Implementation:

You can now test the API with different pagination and sorting parameters:

- **GET /api/products?page=0&size=10:** Get the first 10 products.
- **GET /api/products?page=1&size=5&sort=price,desc:** Get the second page of 5 products, sorted by price in descending order.
- **GET /api/products?sort=category,asc&sort=name,desc:** Get all products sorted first by category (ascending) and then by name (descending).

Page vs. Slice:

Spring Data also provides a `Slice<T>` return type for paginated results. The main difference is that `Page` knows the total number of elements and pages, which requires an additional count query. `Slice` only knows if there is a next slice, making it more efficient if you don't need the total count.

Criteria API

The JPA Criteria API is a programmatic way to build type-safe queries. Instead of writing JPQL or SQL strings, you use Java objects and methods to define the query structure.

Why do we need the Criteria API?

While query methods and the `@Query` annotation are sufficient for static queries, they are not ideal for building dynamic queries where the query structure changes based on runtime conditions.

Consider an e-commerce site with advanced search filters: users can filter products by category, price range, brand, rating, etc. The combination of these filters can vary with each search. Building a JPQL string for every possible combination is cumbersome and error-prone.

The Criteria API shines in these scenarios. It allows you to build a query programmatically, adding constraints and conditions dynamically based on user input.

Real-world Implementation (Amazon Search Filters):

When you search on Amazon, you get a sidebar with numerous filter options. Each time you check or uncheck a filter, a new query is sent to the backend. The backend service needs to dynamically construct a query that includes all the selected filters. The Criteria API is perfectly suited for this task.

Advantages of the Criteria API:

- **Type-Safety:** Queries are constructed using Java code, so the compiler can catch errors at compile time rather than at runtime.
- **Dynamic Queries:** It's easy to add clauses to a query dynamically.
- **Readability and Maintainability:** For complex queries, a programmatic approach can be easier to read and maintain than a long, concatenated JPQL string.
- **Reduced Risk of Injection Attacks:** As you are not concatenating strings, the risk of JPQL injection is minimised.

Breaking Down the Criteria API

The Criteria API has three main components:

1. **CriteriaBuilder:** This is the factory for all query elements. You use it to create CriteriaQuery objects and to build expressions, predicates, and orderings. You obtain a CriteriaBuilder from the EntityManager.
2. **CriteriaQuery:** This object represents the query itself. You define what you want to select, from where, and any filtering or ordering.
3. **Root:** A Root object represents the entity you are querying from. It's the starting point for navigation and for creating path expressions to access entity attributes.

Basic Query Structure:

A typical Criteria API query involves these steps:

1. Get an instance of CriteriaBuilder.
2. Create a CriteriaQuery object.
3. Get a Root object for the entity you want to query.
4. Specify what to select using criteriaQuery.select().

5. Add WHERE clauses using `criteriaQuery.where()` and predicates.
6. Create the Query object from the CriteriaQuery.
7. Execute the query.

Example: Basic SQL Query using Criteria API

Let's replicate `SELECT p FROM Product p WHERE p.category = 'Electronics'` using the Criteria API.

```
EntityManager em = // obtain from persistence context

CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Product> cq = cb.createQuery(Product.class);

Root<Product> product = cq.from(Product.class);
cq.select(product);
cq.where(cb.equal(product.get("category"), "Electronics"));

TypedQuery<Product> query = em.createQuery(cq);
List<Product> products = query.getResultList();
```

Key Components in the Example:

- **`cb.createQuery(Product.class)`**: Creates a CriteriaQuery that will return Product objects.
- **`cq.from(Product.class)`**: Specifies that the query is from the Product entity.
- **`product.get("category")`**: A path expression to get the category attribute of the Product entity.
- **`cb.equal(...)`**: A predicate that represents an equality condition.

Building Dynamic Queries

The true power of the Criteria API lies in its ability to build queries dynamically. Let's create a dynamic search for products based on optional criteria: name, category, and a minimum price.

Product Search DAO:

It's a good practice to create a separate DAO or repository implementation for complex queries.

```
@Repository
public class ProductSearchDao {

    private EntityManager entityManager;
```

```
public List<Product> findProductsByCriteria(String name, String
category, Double minPrice) {
    CriteriaBuilder cb = entityManager.getCriteriaBuilder();
    CriteriaQuery<Product> cq = cb.createQuery(Product.class);

    Root<Product> product = cq.from(Product.class);
    List<Predicate> predicates = new ArrayList<>();

    if (name != null && !name.isEmpty()) {
        predicates.add(cb.like(product.get("name"), "%" + name + "%"));
    }
    if (category != null && !category.isEmpty()) {
        predicates.add(cb.equal(product.get("category"), category));
    }
    if (minPrice != null) {
        predicates.add(cb.greaterThanOrEqualTo(product.get("price"),
minPrice));
    }

    cq.where(predicates.toArray(new Predicate[0]));

    TypedQuery<Product> query = entityManager.createQuery(cq);
    return query.getResultList();
}
```

Explanation:

1. We create a List<Predicate> to hold our WHERE conditions.
2. For each search parameter that is not null or empty, we create a corresponding Predicate and add it to the list.
 - **cb.like():** For partial string matching.
 - **cb.equal():** For an exact match.
 - **cb.greaterThanOrEqualTo():** For a numeric comparison.
3. Finally, we combine all the predicates using cq.where(). The toArray() method converts the list of predicates into an array, which is then passed to the where method. If the predicates list is empty, no WHERE clause is added.

This approach is much cleaner and safer than string concatenation for building dynamic SQL queries.

Common DAO

DAO

DAO (Data Access Object) is a design pattern used to abstract and encapsulate all access to a data source. In Spring Boot, while JpaRepository already offers built-in CRUD operations, creating a Common DAO allows you to:

- Reuse generic CRUD logic.
- Add logging, exception handling, or auditing in a centralised way.
- Reduce boilerplate in service classes.
- Make your codebase cleaner and more maintainable.

In many applications, you'll find yourself writing the same basic CRUD (Create, Read, Update, Delete) methods for different entities. A Common DAO or Generic DAO is a design pattern that aims to eliminate this boilerplate code by providing a generic implementation of common data access operations.

Benefits of a Common DAO:

- **Code Reusability:** Write the common data access logic once and reuse it for all your entities.
- **Reduced Boilerplate:** No need to write a separate repository for each entity if you only need basic CRUD operations.
- **Consistency:** Ensures that common operations are implemented consistently across the application.

Example: Product Management System

Let's create a generic Common DAO setup for managing Product entities.

Product.java – The Entity

```
package com.example.demo.model;
import jakarta.persistence.*;
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private double price;
    // Getters and setters
}
```

```
}
```

ProductRepository.java

```
package com.example.demo.repository;
import com.example.demo.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

CommonDao<T, ID> – Generic Interface

```
package com.example.demo.dao;
import java.util.List;
import java.util.Optional;
public interface CommonDao<T, ID> {
    T save(T entity);
    Optional<T> findById(ID id);
    List<T> findAll();
    void deleteById(ID id);
    void delete(T entity);
}
```

CommonDaoImpl<T, ID> – Generic Implementation

```
package com.example.demo.dao;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;
public class CommonDaoImpl<T, ID> implements CommonDao<T, ID> {
    private static final Logger logger =
LoggerFactory.getLogger(CommonDaoImpl.class);
    private final JpaRepository<T, ID> repository;

    public CommonDaoImpl(JpaRepository<T, ID> repository) {
        this.repository = repository;
    }
    @Override
    public T save(T entity) {
        try {
            logger.info("Saving entity: {}", entity);
            return repository.save(entity);
        }
    }
}
```

```
        } catch (Exception e) {
            logger.error("Save failed for entity {}: {}", entity,
e.getMessage());
            throw e;
        }
    }

    @Override
    public Optional<T> findById(ID id) {
        return repository.findById(id);
    }

    @Override
    public List<T> findAll() {
        try {
            List<T> list = repository.findAll();
            logger.info("Fetched {} records", list.size());
            return list;
        } catch (Exception e) {
            logger.error("Error fetching all: {}", e.getMessage());
            throw e;
        }
    }

    @Override
    public void deleteById(ID id) {
        repository.deleteById(id);
    }

    @Override
    public void delete(T entity) {
        repository.delete(entity);
    }
}
```

ProductService.java – Using the CommonDao

```
package com.example.demo.service;
import com.example.demo.dao.CommonDao;
import com.example.demo.dao.CommonDaoImpl;
import com.example.demo.model.Product;
import com.example.demo.repository.ProductRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ProductService {
    private final CommonDao<Product, Long> productDao;
```

```
public ProductService(ProductRepository productRepository) {
    this.productDao = new CommonDaoImpl<>(productRepository); // Inject
repository into DAO
}

public Product createProduct(Product product) {
    return productDao.save(product);
}

public List<Product> getAllProducts() {
    return productDao.findAll();
}

public void deleteProduct(Long id) {
    productDao.deleteById(id);
}
}
```

Using a Generic/Common DAO in Spring Boot allows for:

- Centralized data access handling.
- Cleaner service layers.
- Enhanced testability and logging.

This pattern is especially helpful in large applications with many entities where CRUD logic is similar across multiple models.

Conclusion

We have covered several advanced topics in Spring Data JPA that are essential for building robust, scalable, and maintainable data access layers in modern Java applications.

Key Takeaways:

- Pagination and Sorting are crucial for performance and user experience when dealing with large datasets. The `PagingAndSortingRepository` and the `Pageable` interface in Spring Data make their implementation straightforward.
- The Criteria API is the go-to solution for building dynamic, type-safe queries. It provides a powerful and flexible alternative to string-based JPQL or SQL queries, especially for complex search functionalities.
- The Common DAO pattern is a useful technique for reducing boilerplate code.

By mastering these advanced concepts, you can leverage the full power of Spring Data JPA to build sophisticated and efficient data-driven applications.

Instructor Codes

1. [Directory Application](#)

References

1. [Pagination & Sorting](#)
2. [Peagable](#)
3. [Criteria Query](#)